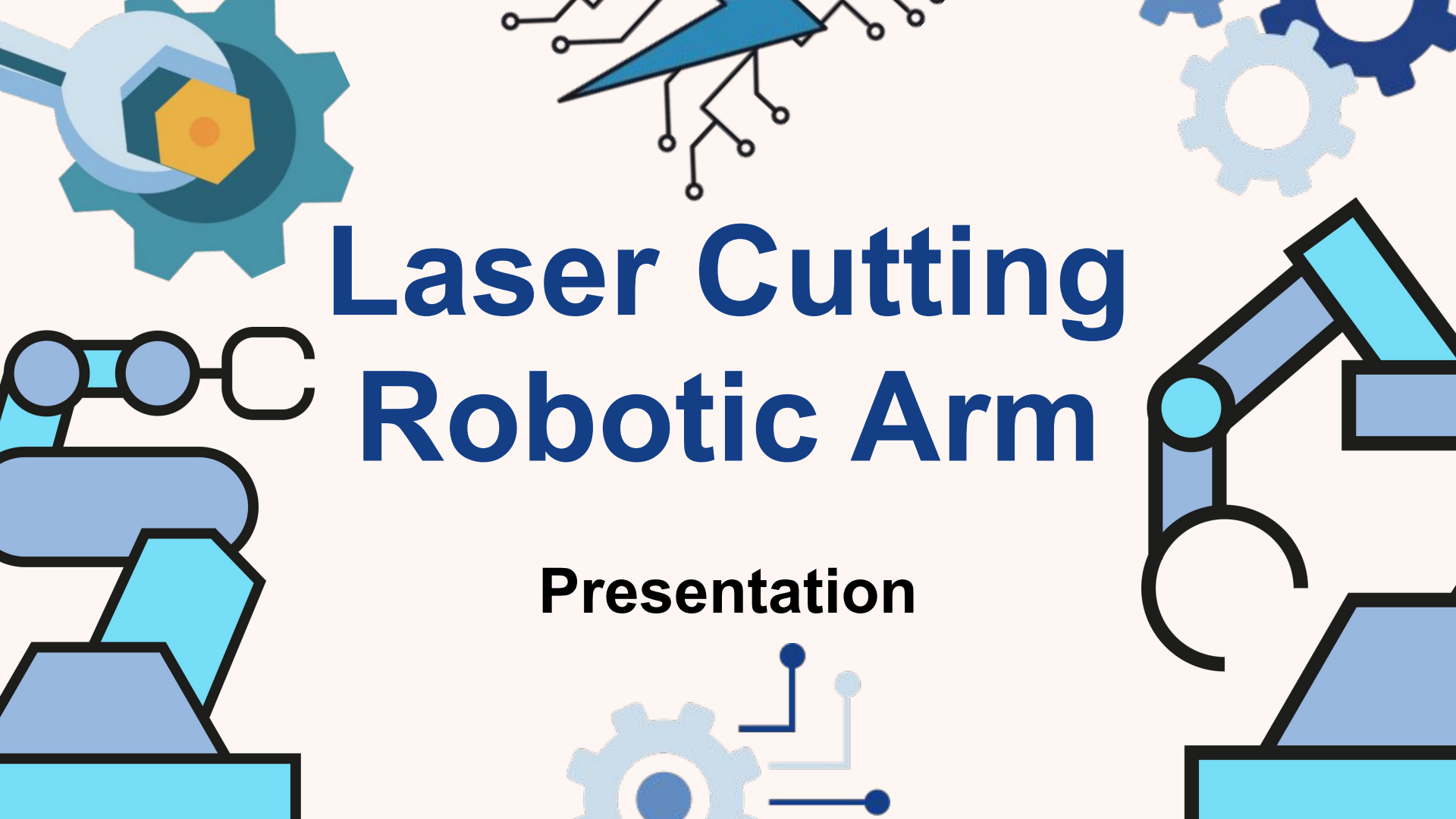
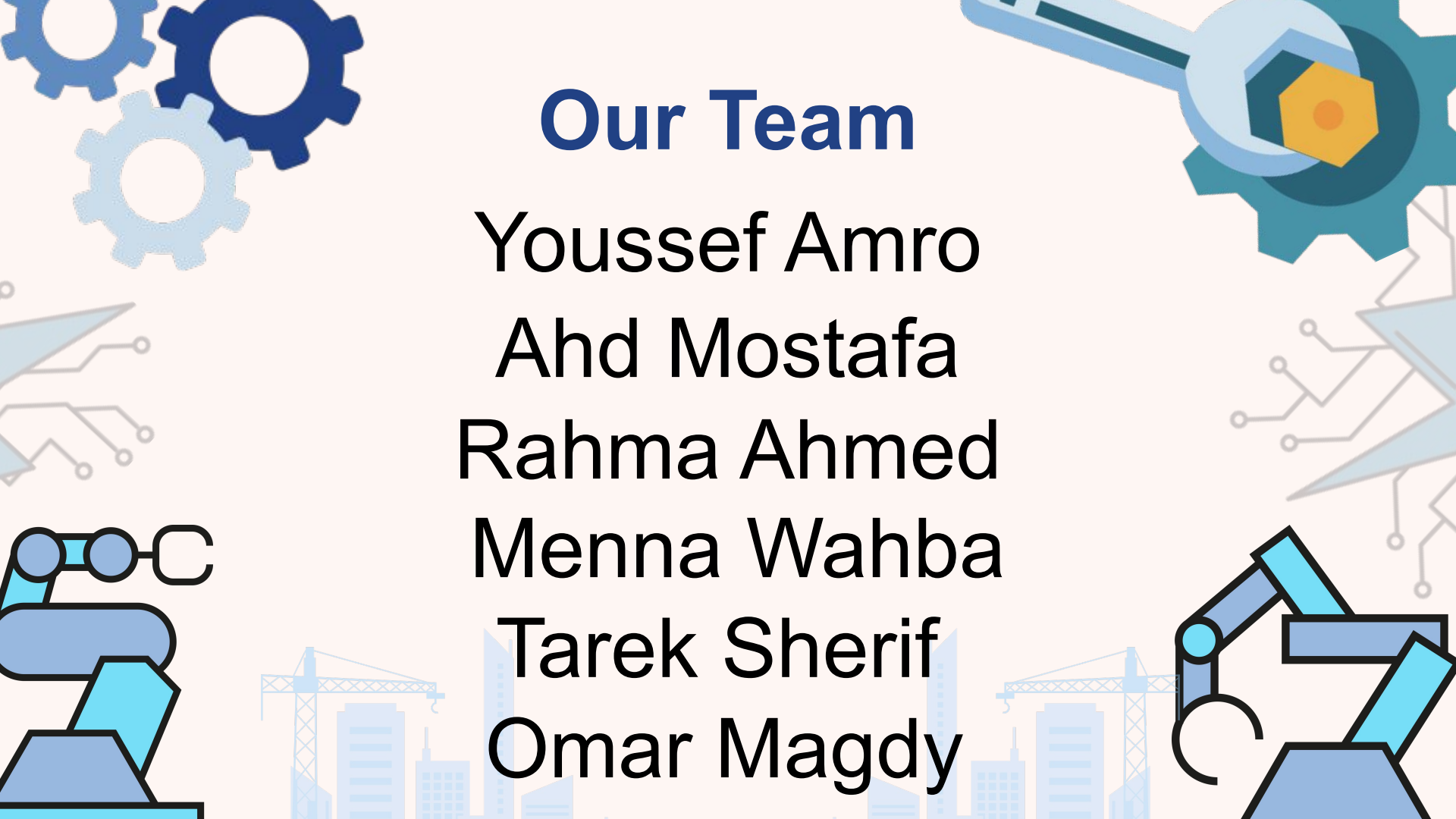




Laser Cutting Robotic Arm

Presentation



Our Team

Youssef Amro

Ahd Mostafa

Rahma Ahmed

Menna Wahba

Tarek Sherif

Omar Magdy

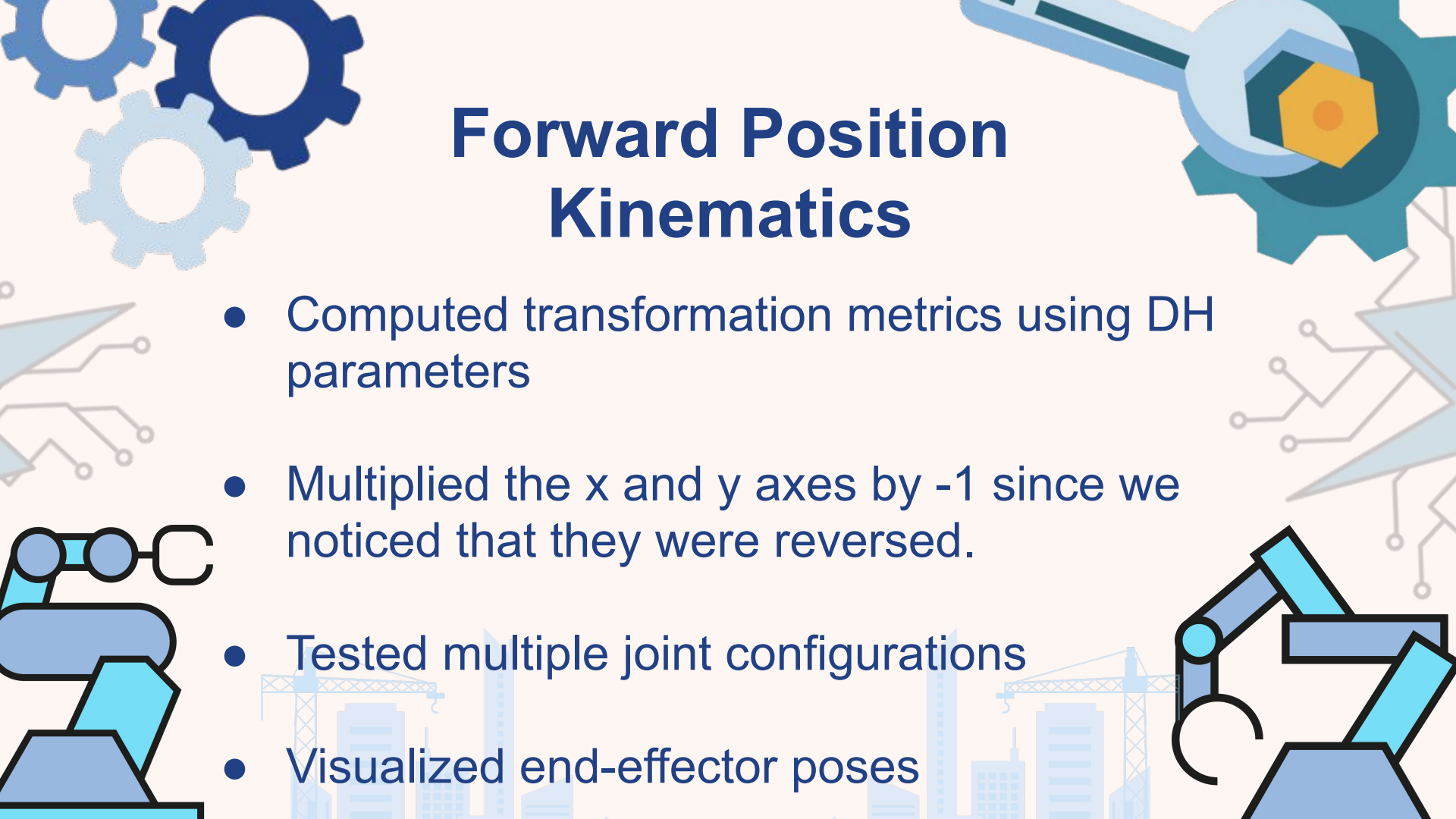


Content

- **DH Convention**
- **Forward & Inverse Position Kinematics**
- **Jacobian Matrix**
- **Forward & Inverse Velocity Kinematics**
- **Forward & Inverse Acceleration Kinematics**
- **Task & joint space Trajectory**
- **Fuzzy Logic controller**
- **Closed loop Trajectory**

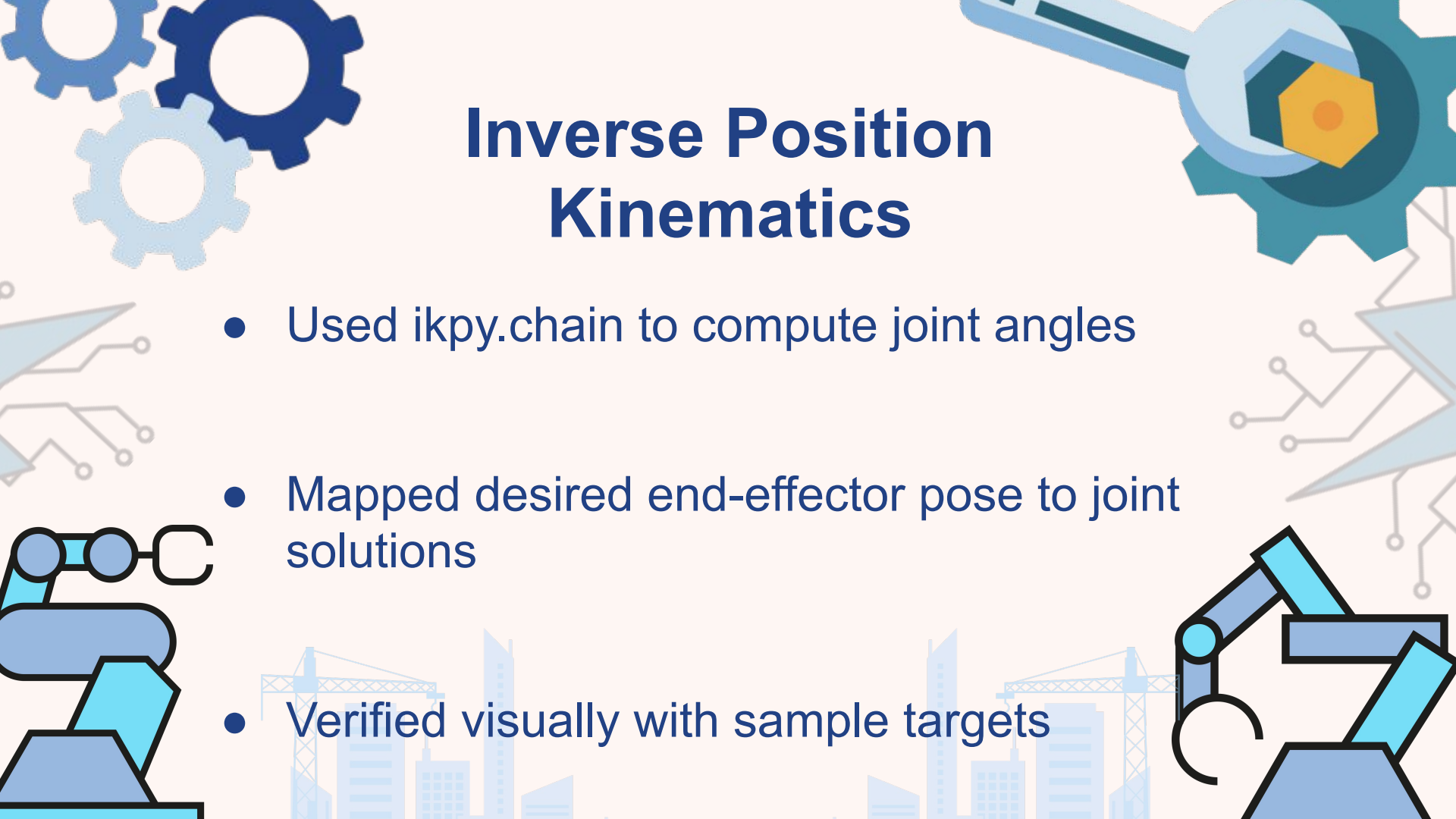
DH Convention

- Defined X – Z frames for each joint
- Constructed DH parameter table (q_i, d_i, a_i, α_i)
- Frames drawn on two different robot views
- Parameters used for kinematic model



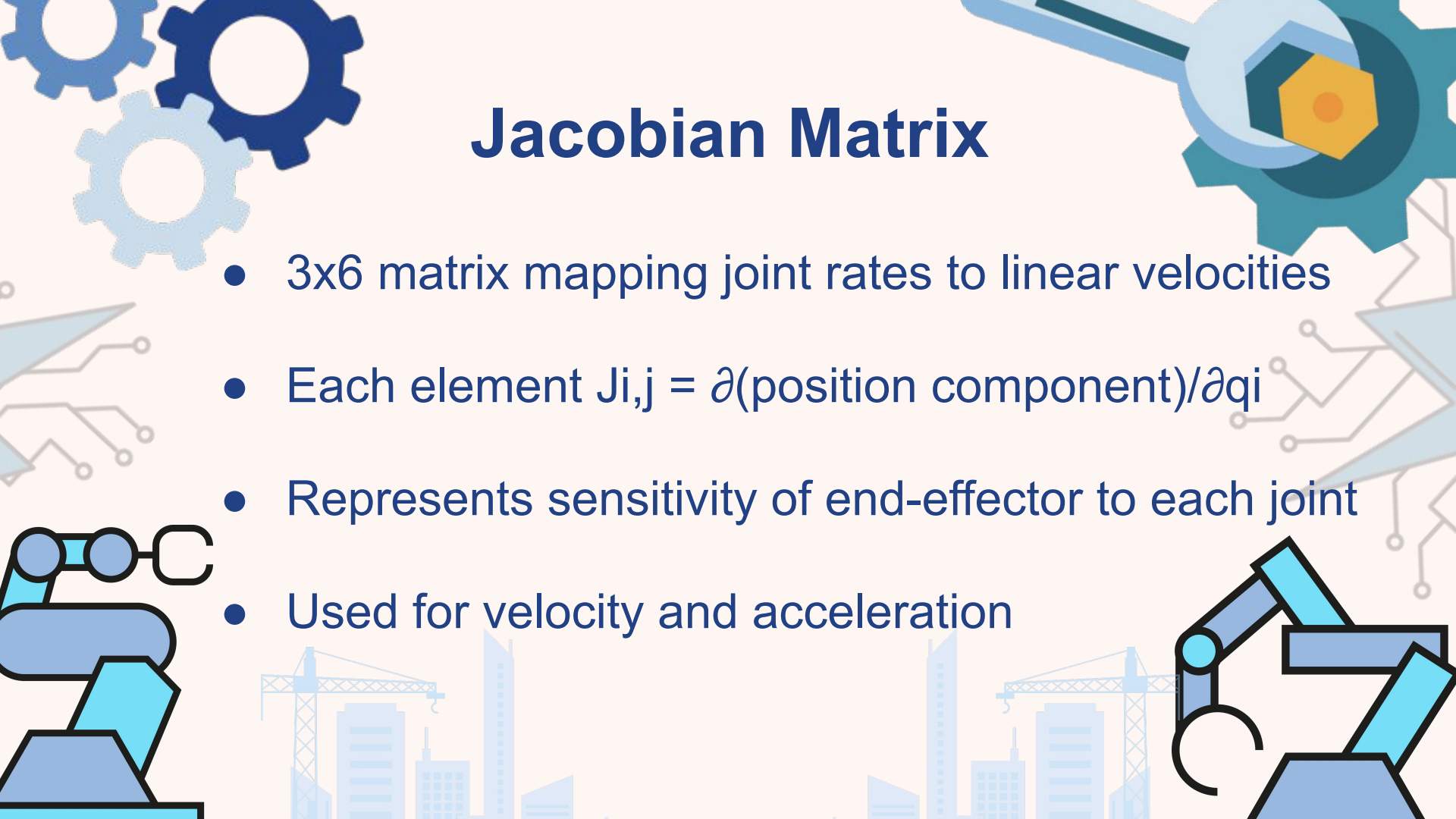
Forward Position Kinematics

- Computed transformation metrics using DH parameters
- Multiplied the x and y axes by -1 since we noticed that they were reversed.
- Tested multiple joint configurations
- Visualized end-effector poses



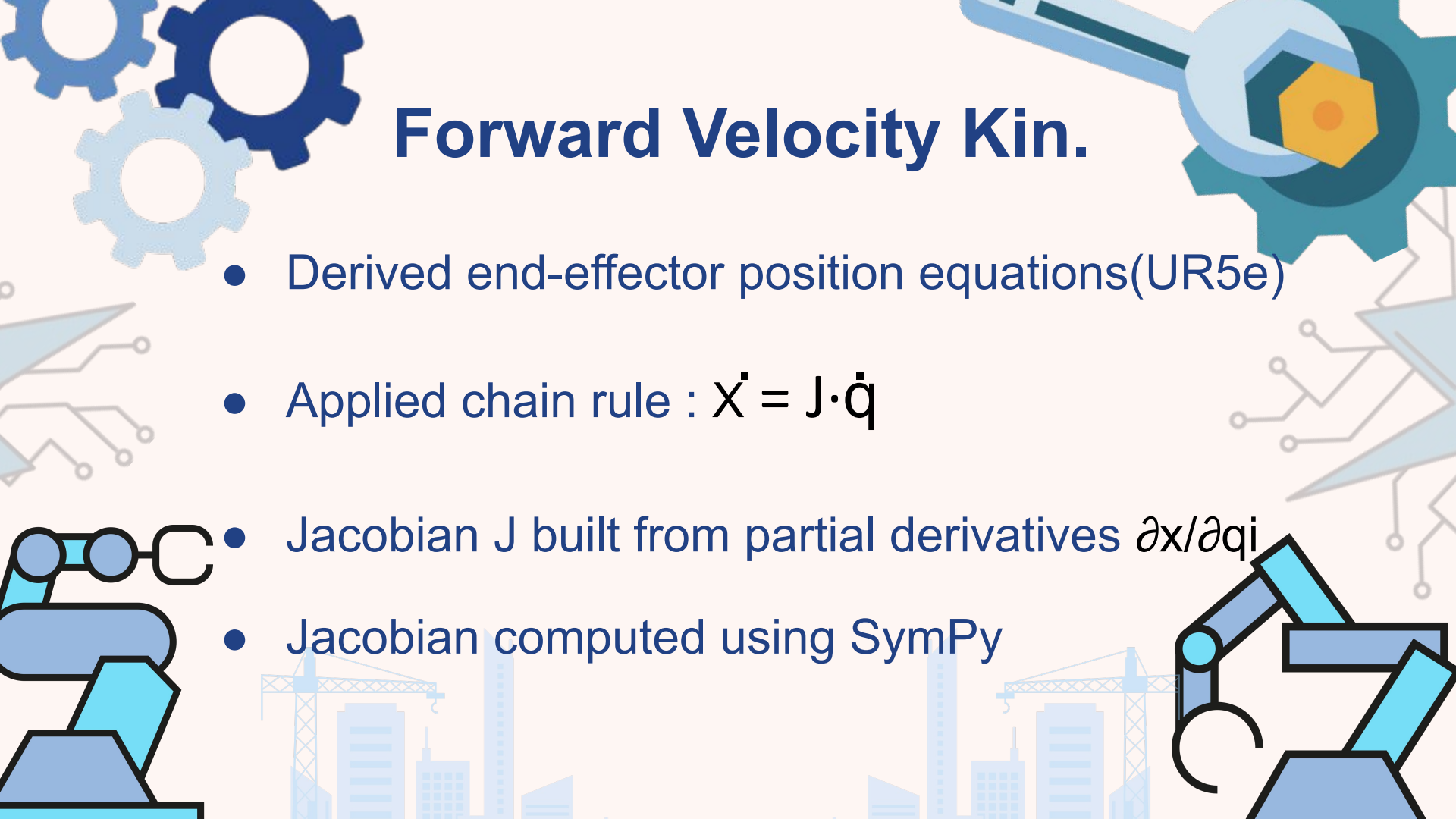
Inverse Position Kinematics

- Used ikpy.chain to compute joint angles
- Mapped desired end-effector pose to joint solutions
- Verified visually with sample targets



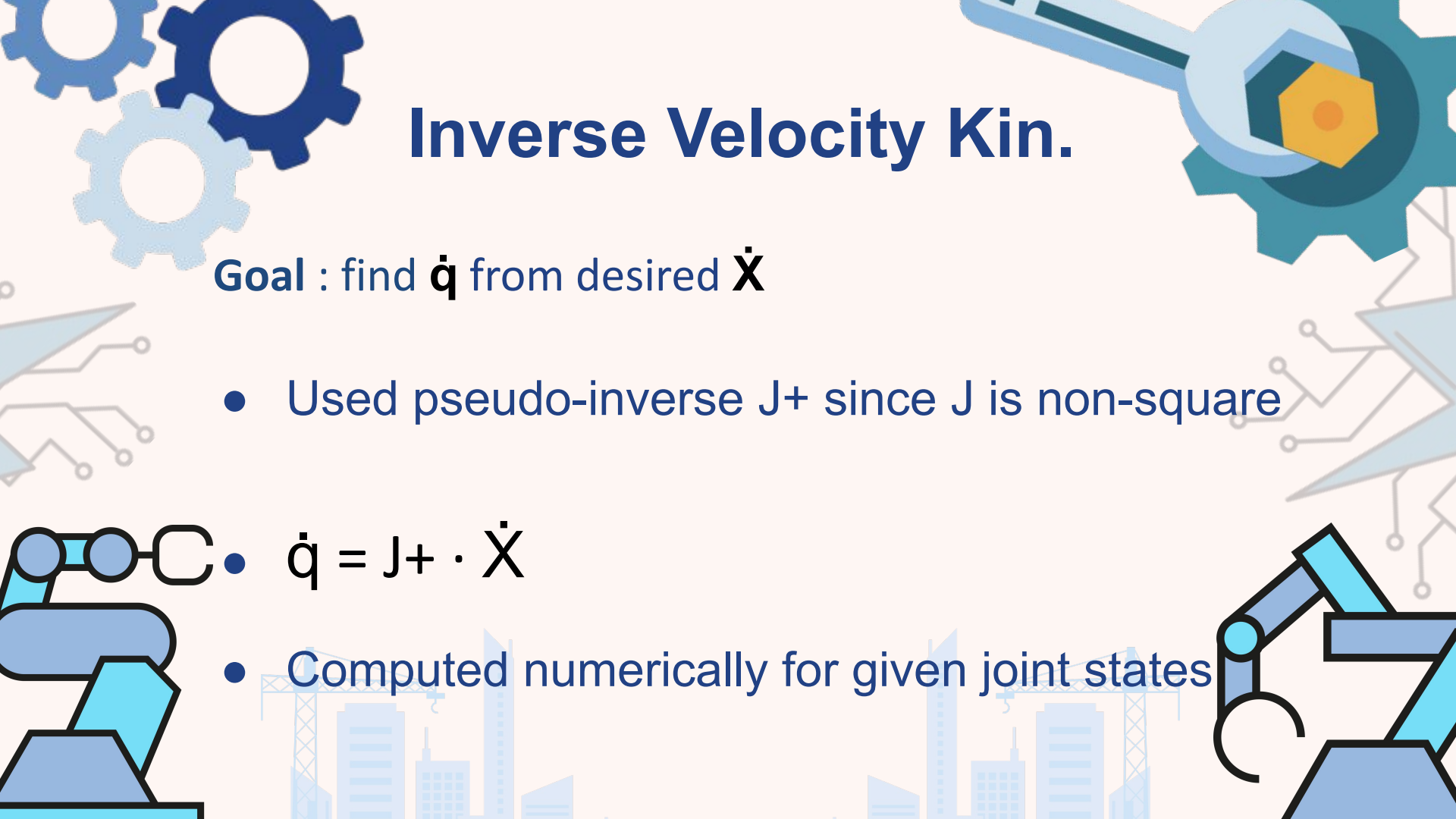
Jacobian Matrix

- 3x6 matrix mapping joint rates to linear velocities
- Each element $J_{i,j} = \partial(\text{position component})/\partial q_i$
- Represents sensitivity of end-effector to each joint
- Used for velocity and acceleration



Forward Velocity Kin.

- Derived end-effector position equations (UR5e)
- Applied chain rule : $\dot{X} = J \cdot \dot{q}$
- Jacobian J built from partial derivatives $\partial x / \partial q_i$
- Jacobian computed using SymPy



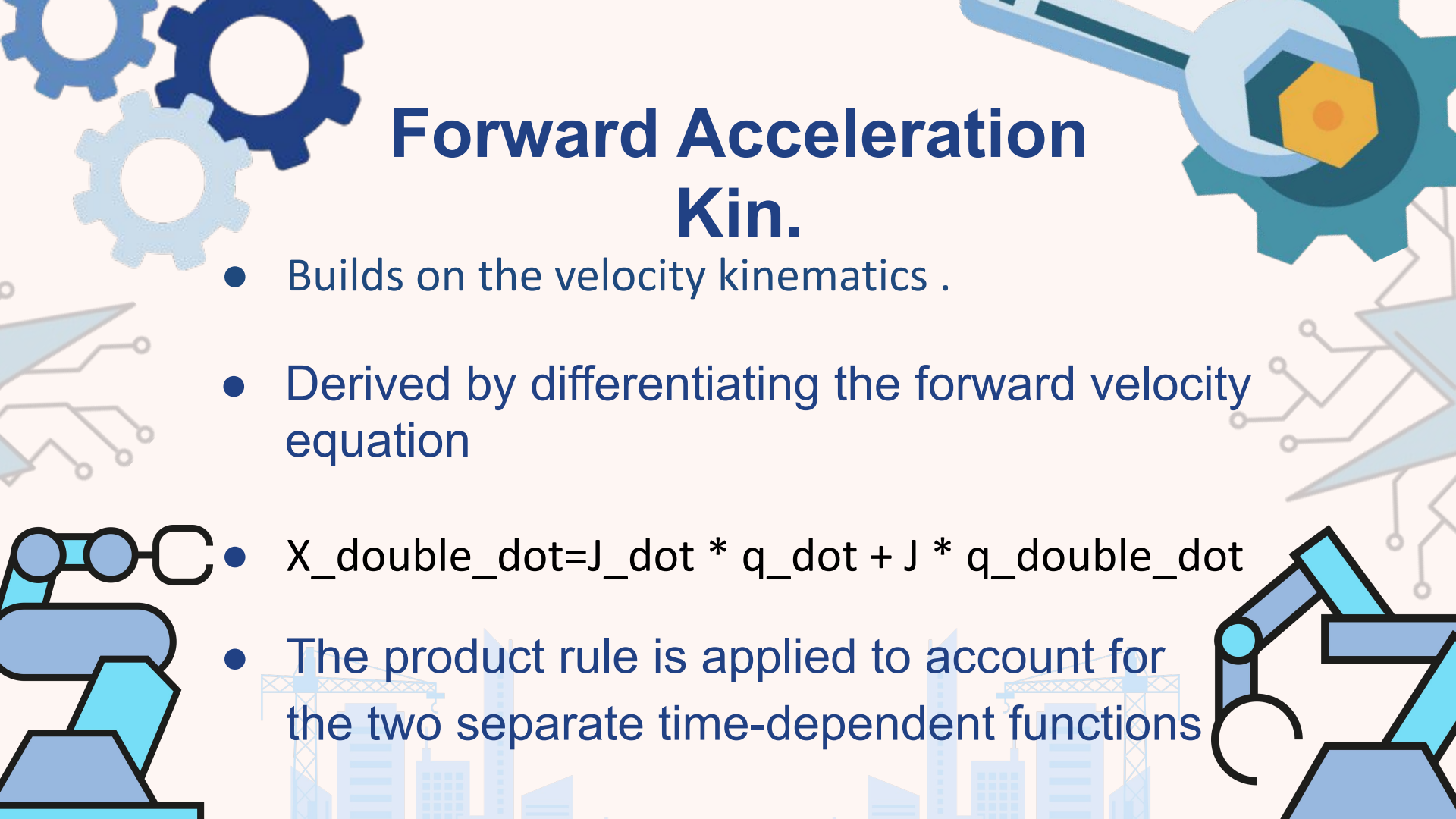
Inverse Velocity Kin.

Goal : find $\dot{\mathbf{q}}$ from desired $\dot{\mathbf{X}}$

- Used pseudo-inverse J^+ since J is non-square

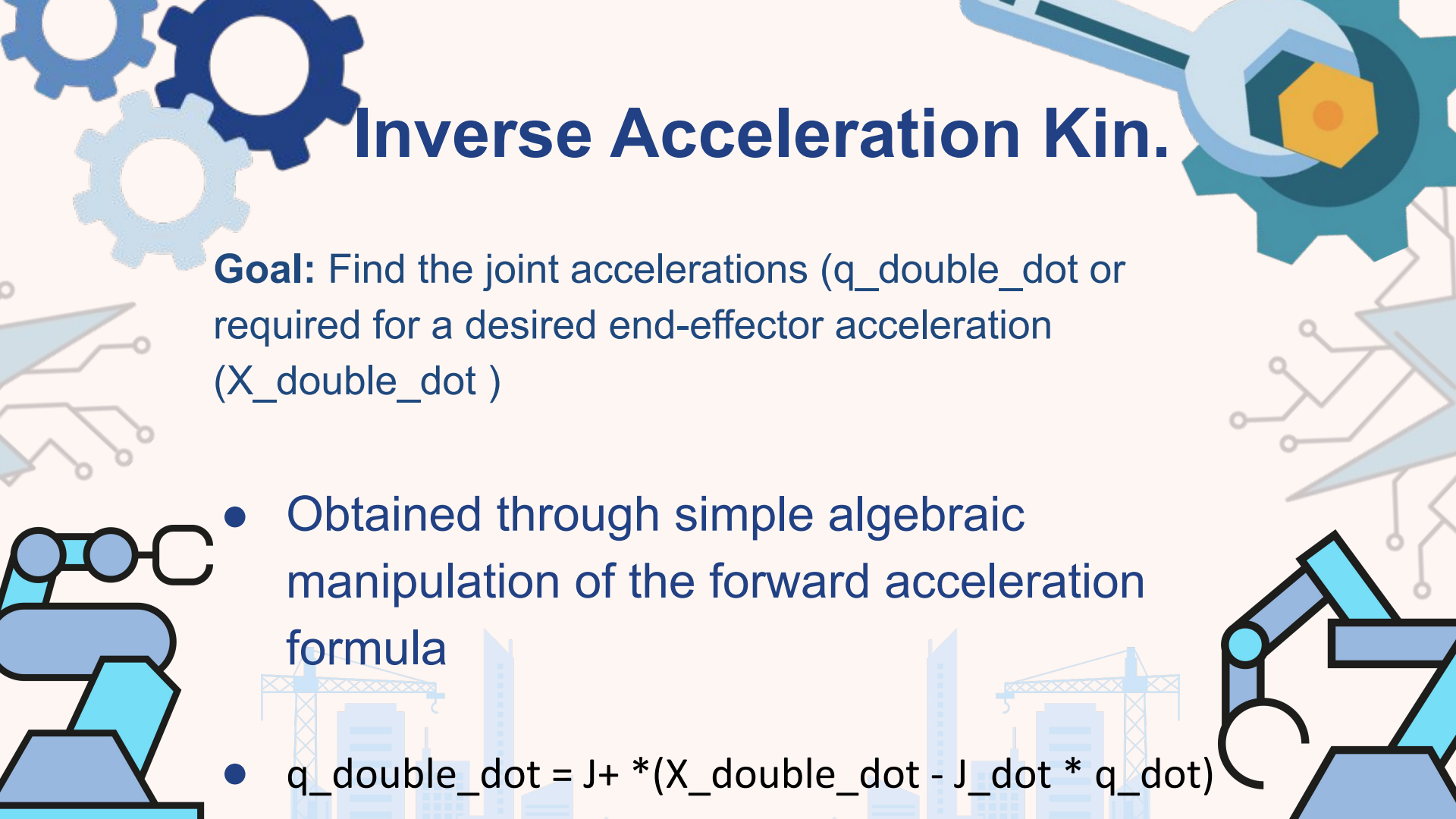
- $\dot{\mathbf{q}} = J^+ \cdot \dot{\mathbf{X}}$

- Computed numerically for given joint states



Forward Acceleration Kin.

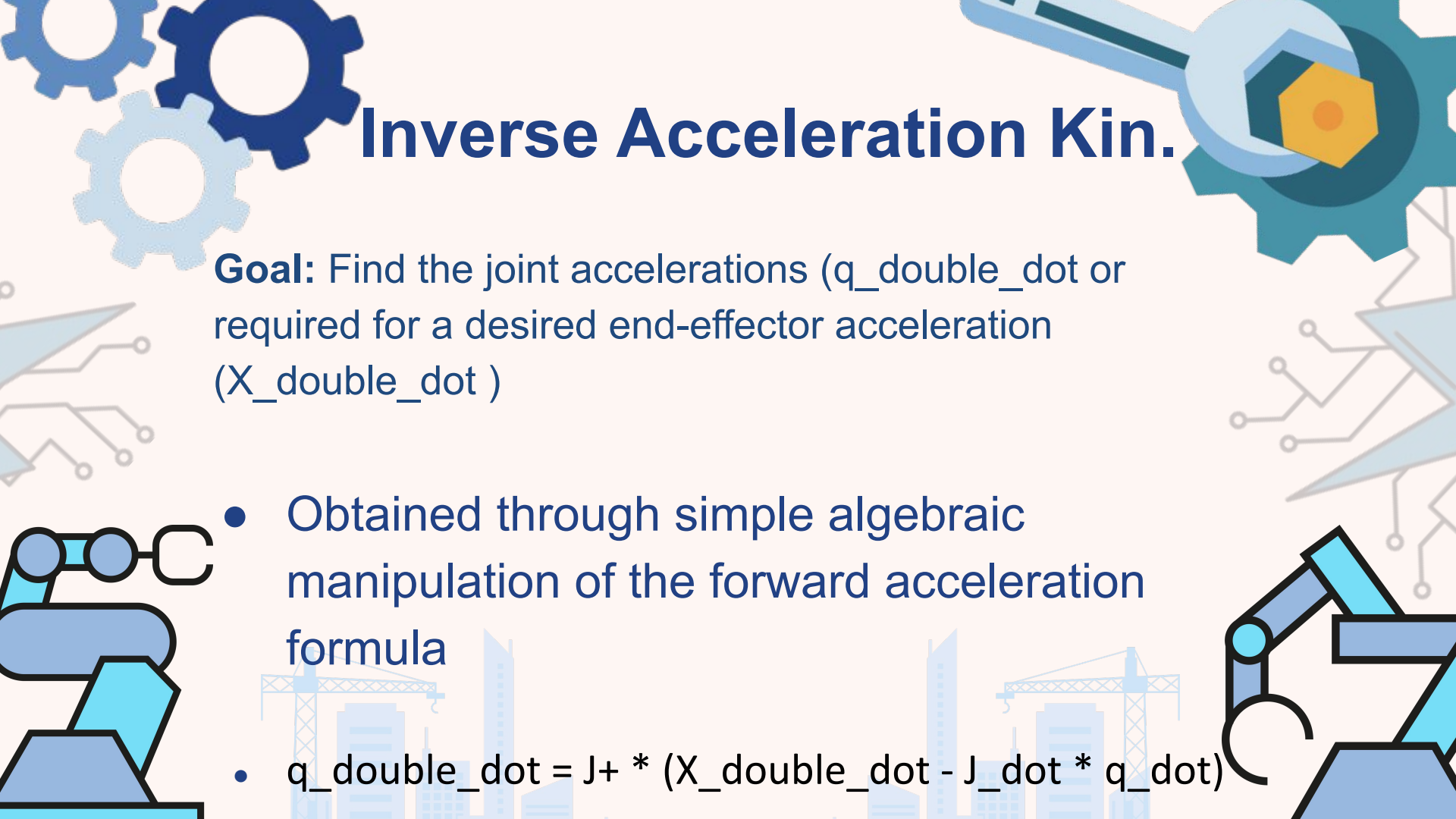
- Builds on the velocity kinematics .
- Derived by differentiating the forward velocity equation
- $\ddot{X} = \dot{J} * \dot{q} + J * \ddot{q}$
- The product rule is applied to account for the two separate time-dependent functions



Inverse Acceleration Kin.

Goal: Find the joint accelerations ($\ddot{q}$ or required for a desired end-effector acceleration ($\ddot{X}$)

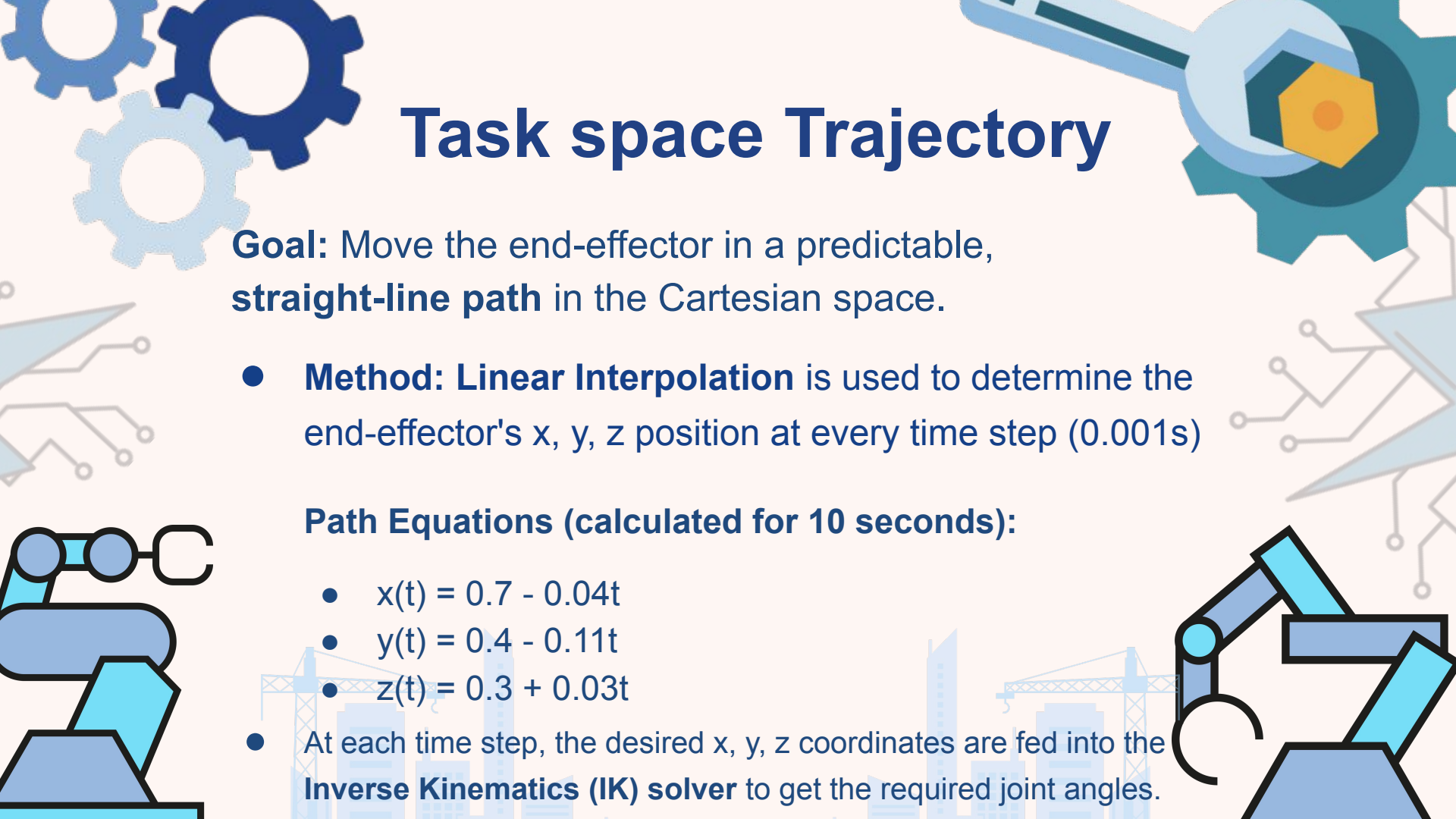
- Obtained through simple algebraic manipulation of the forward acceleration formula
- $$\ddot{q} = J^+ (\ddot{X} - \dot{J} \dot{q})$$



Inverse Acceleration Kin.

Goal: Find the joint accelerations ($\ddot{q}$ or required for a desired end-effector acceleration ($\ddot{X}$)

- Obtained through simple algebraic manipulation of the forward acceleration formula
- $\ddot{q} = J^+ * (\ddot{X} - \dot{J} * \dot{q})$



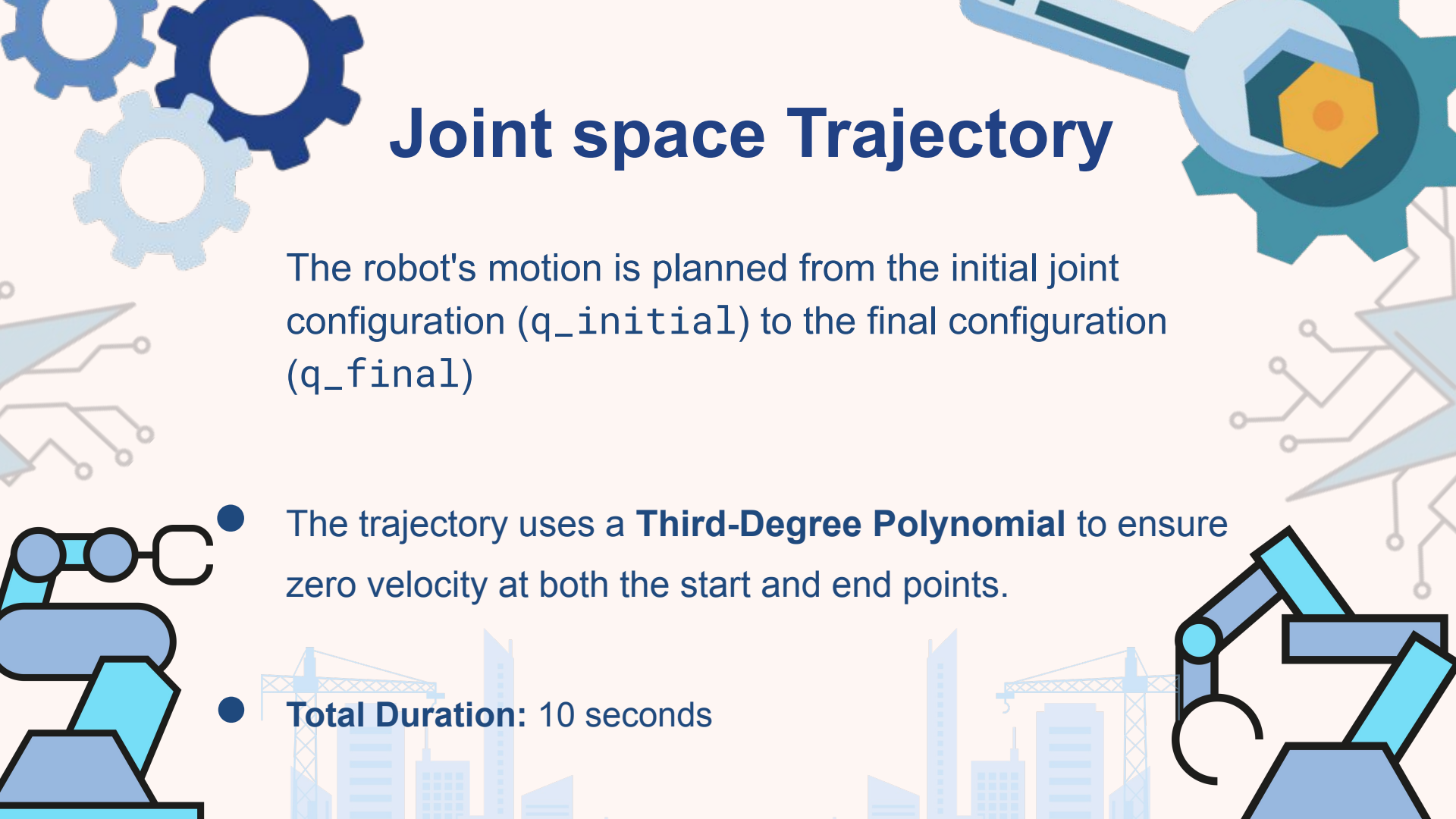
Task space Trajectory

Goal: Move the end-effector in a predictable, **straight-line path** in the Cartesian space.

- **Method: Linear Interpolation** is used to determine the end-effector's x, y, z position at every time step (0.001s)

Path Equations (calculated for 10 seconds):

- $x(t) = 0.7 - 0.04t$
- $y(t) = 0.4 - 0.11t$
- $z(t) = 0.3 + 0.03t$
- At each time step, the desired x, y, z coordinates are fed into the **Inverse Kinematics (IK) solver** to get the required joint angles.



Joint space Trajectory

The robot's motion is planned from the initial joint configuration (q_{initial}) to the final configuration (q_{final})

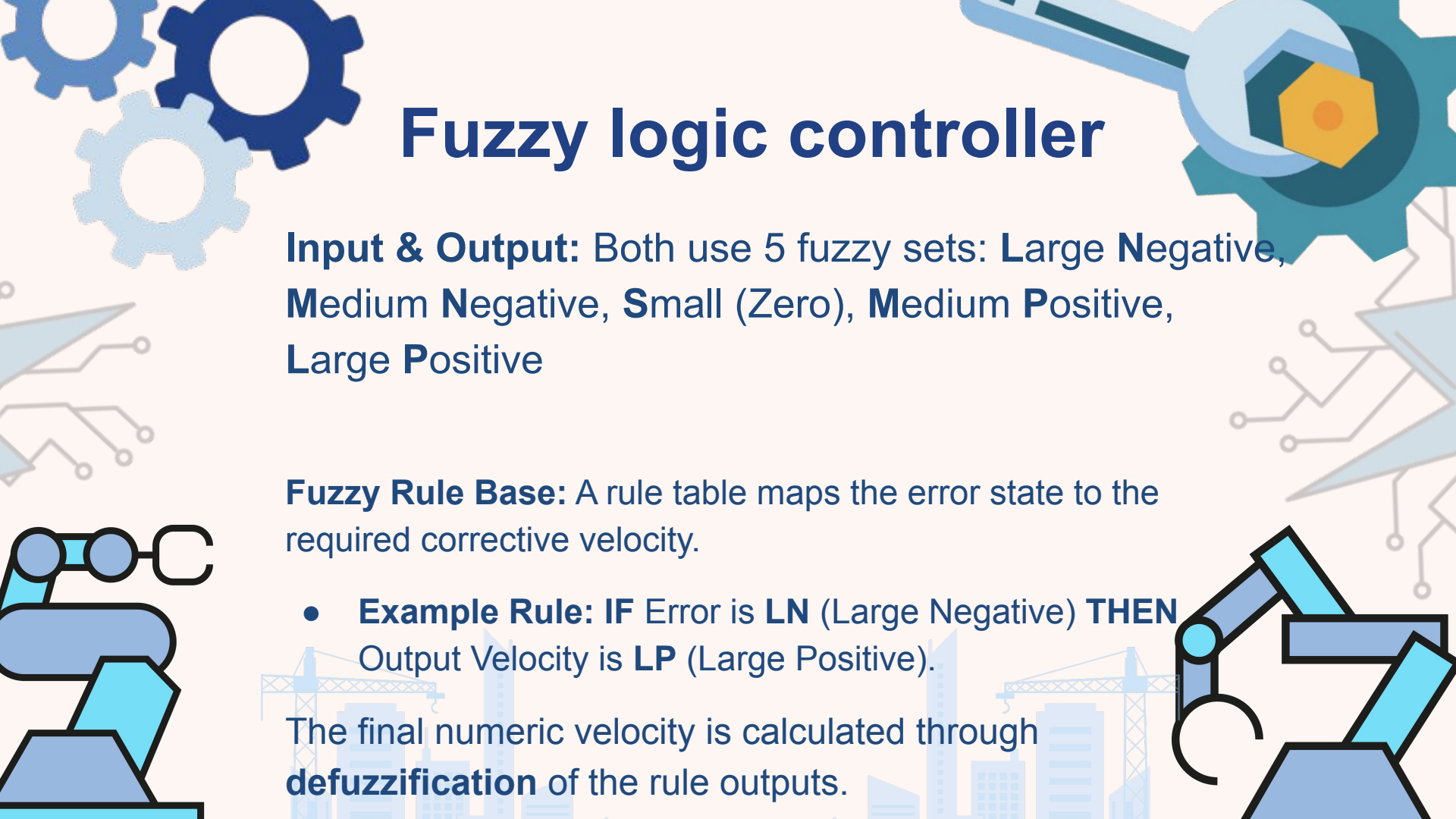
- The trajectory uses a **Third-Degree Polynomial** to ensure zero velocity at both the start and end points.
- **Total Duration:** 10 seconds



Fuzzy logic controller

Goal: Ensure accurate trajectory tracking by continuously correcting position errors in a **closed-loop system**

- **Mechanism:** FLC converts a numeric position error (e) into a corrective velocity command (v)
- The output velocity (v) is always applied in the opposite direction of the position deviation



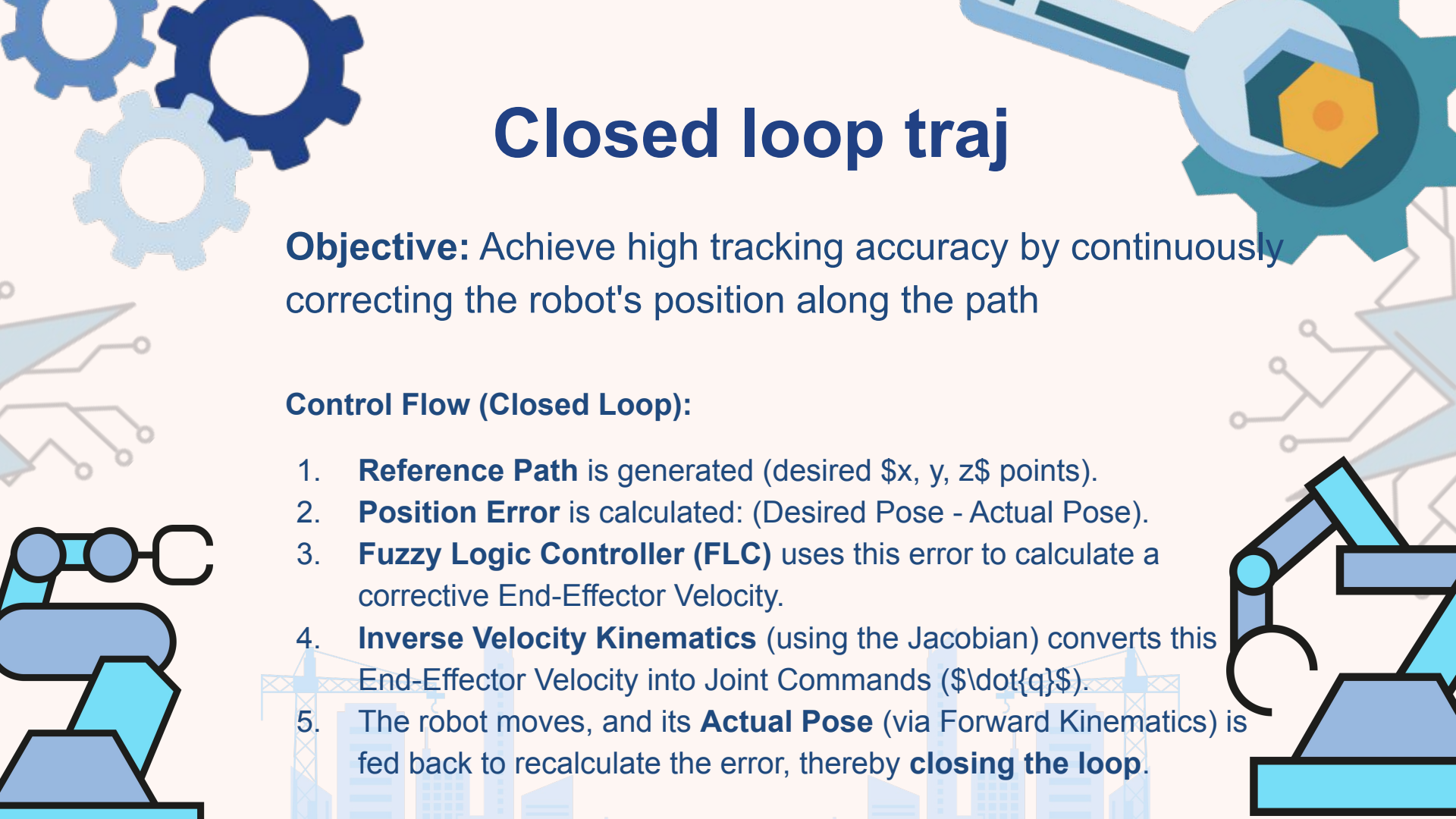
Fuzzy logic controller

Input & Output: Both use 5 fuzzy sets: **Large Negative**, **Medium Negative**, **Small (Zero)**, **Medium Positive**, **Large Positive**

Fuzzy Rule Base: A rule table maps the error state to the required corrective velocity.

- **Example Rule:** IF Error is **LN** (Large Negative) THEN Output Velocity is **LP** (Large Positive).

The final numeric velocity is calculated through **defuzzification** of the rule outputs.



Closed loop traj

Objective: Achieve high tracking accuracy by continuously correcting the robot's position along the path

Control Flow (Closed Loop):

1. **Reference Path** is generated (desired x, y, z points).
2. **Position Error** is calculated: (Desired Pose - Actual Pose).
3. **Fuzzy Logic Controller (FLC)** uses this error to calculate a corrective End-Effector Velocity.
4. **Inverse Velocity Kinematics** (using the Jacobian) converts this End-Effector Velocity into Joint Commands ($\dot{q}$).
5. The robot moves, and its **Actual Pose** (via Forward Kinematics) is fed back to recalculate the error, thereby **closing the loop**.



**Thank
You**